	ISIS 4218 Concurrencia, Paralelismo y Distribución	Entregable Tarea 3	Fecha 2026-sep-25

Índice

[Presentado por](#)
[1. Introducción](#)
- [1. Archivos Relevantes en el .ZIP](#)
- [2. Cambios respecto a Tarea 2](#)
- [3. Modelo STM](#)
- [4. Seguridad y consistencia](#)
- [5. Criterios de evaluación](#)
- [6. Conclusión](#)

Presentado por

Andrés Felipe Peña Castro (código 201618814)

1. Introducción

Esta tarea implementa la misma plataforma de reserva y venta de asientos de la tarea anterior utilizando Software Transactional Memory (`STM`) como modelo de concurrencia. Se mantiene la separación entre las reglas de negocio y el mecanismo de sincronización, concentrando el cambio principalmente en `Gate.Sync`.

A diferencia de la implementación `Threads + Mutex` de la tarea anterior, las operaciones concurrentes sobre un mismo sector pueden calcular sus transiciones sobre versiones locales del estado y validar el resultado al momento del commit. De esta forma, los conflictos se detectan en lugar de evitarse mediante la serialización de toda la operación. La implementación conserva el contrato funcional de la plataforma, incluyendo reservas `all-or-nothing`, confirmaciones, cancelaciones, expiración de holds y snapshots.

1. Archivos Relevantes en el .ZIP

ISIS4218_T03.zip
.... reporte_tarea_3_cpd.pdf
.... a1_threads/
..... README.md
..... .gate_model *-> para esta tarea es “stm”*
..... lib/
..... gate/
..... api.ex
..... row.ex
..... sector.ex *-> ajustes relacionados con la expiración de holds*
..... sync.ex *-> **implementación principal de esta tarea***
..... venue.ex *-> ajuste del manejo del estado después de expiraciones*
..... venue_spec.ex

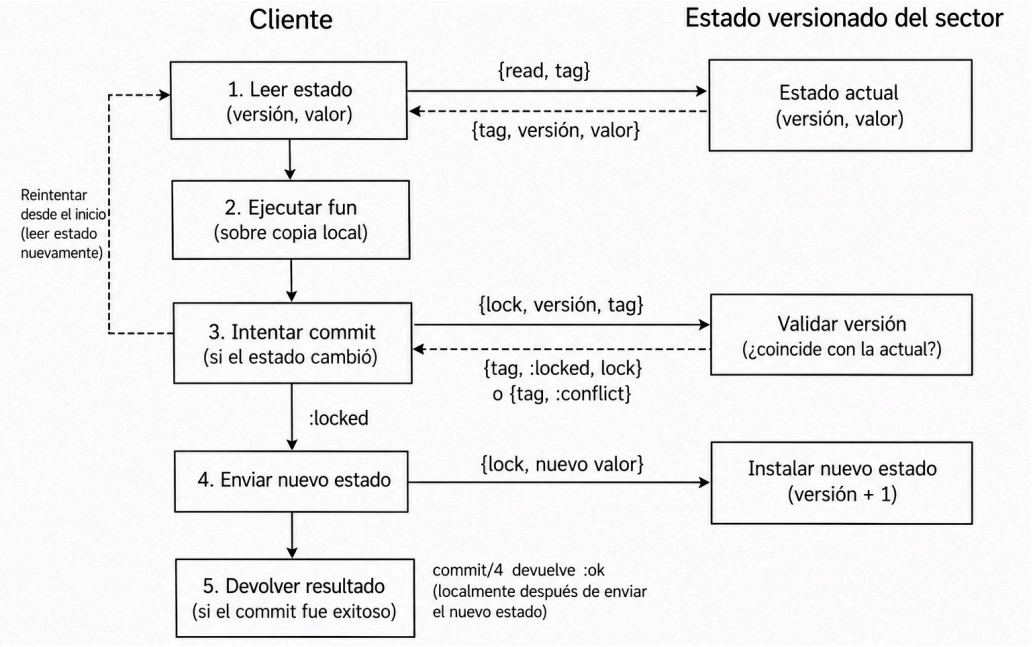
2. Cambios respecto a Tarea 2

Aspecto	Tarea 2, Threads + Mutex	Tarea 3, STM
Modelo	Threads + Mutex	Software Transactional Memory
Estado	Estado del sector dentro de su proceso	Estado del sector + versión
Unidad de conflicto	Sector	Sector
Lectura	El proceso propietario lee	La transacción lee una versión
Ejecución de `fun`	Dentro del proceso propietario	Fuera del estado versionado, sobre una copia local
Concurrencia en un mismo sector	Serializada	Varias transacciones pueden calcular en paralelo
Detección de conflicto	Se evita mediante serialización	Comparación de versiones al hacer commit
Commit	El nuevo estado se instala directamente	Solo se publica si la versión esperada sigue vigente
Conflicto	No hay conflicto explícito	Se descarta el resultado y se reintenta
Diferentes sectores	Independientes	Independientes
Reglas de negocio	`Gate.Sector`	`Gate.Sector`
Mecanismo principal	Mailbox + proceso propietario	Lectura + cálculo local + commit
Parte serializada	Operación completa	Validación y publicación del commit

El cambio principal se concentra en `Gate.Sync` mientras que `Gate.Venue`, `Gate.Sector` y `Gate.Row` mantienen sus responsabilidades, de modo que las reglas de negocio no dependen del modelo de concurrencia utilizado. Esta separación permite reemplazar el mecanismo de sync sin modificar las transiciones que representan operaciones de ticketing.

3. Modelo STM

La implementación utiliza un modelo de Software Transactional Memory (`STM`) basado en estado versionado y concurrencia optimista. Cada sector mantiene su estado junto con una versión que permite detectar modificaciones realizadas por otras transacciones.



Una operación comienza leyendo la versión y el estado actual del sector. Luego ejecuta fun sobre una copia local, sin mantener bloqueado el estado compartido. Si la operación modifica el estado, intenta realizar el `commit` indicando la versión que utilizó.

Durante el `commit` se valida que la versión continúe vigente. Si existe un conflicto, el resultado calculado se descarta y la transacción vuelve a leer el estado actualizado para intentarlo nuevamente. Si la versión es válida, se autoriza el `commit` y se envía el nuevo estado para su instalación, incrementando la versión.

De esta forma, varias transacciones pueden ejecutar su lógica de negocio de manera concurrente sobre copias locales, mientras que la validación y publicación de cambios se realizan de forma controlada. Cuando la operación modifica el estado, el resultado solo se devuelve después de que el nuevo estado ha sido aceptado, evitando que una operación calculada sobre un estado obsoleto produzca efectos en el estado compartido.

4. Seguridad y consistencia

La validación de versiones evita que una transacción calculada sobre un estado obsoleto publique sus cambios, ya que ante un conflicto el intento se descarta y se reintenta sobre el estado actualizado.

La reserva se aplica de **forma atómica**: todas las modificaciones necesarias para asignar los asientos se publican como un único nuevo estado del sector. Por lo tanto, una operación que no puede completarse no deja una reserva parcialmente aplicada.

En los snapshots, la lectura corresponde a una versión concreta del sector, evitando combinar datos de estados diferentes dentro del mismo sector. Las reglas de negocio continúan siendo gestionadas por `Gate.Sector`, manteniendo las condiciones de reserva, confirmación, cancelación y expiración.

5. Criterios de evaluación

Criterio	Peso	Qué evalúa	Resultado de la implementación
Functional	20%	Cumplimiento del contrato de `Gate.API`: reservas, confirmaciones, cancelaciones, disponibilidad, snapshots y manejo de errores.	**13/13 pruebas locales** completadas correctamente, cubriendo las operaciones principales.
Safety	35%	Ausencia de doble venta, pérdida de ventas, estados inconsistentes, errores de revenue y holds que permanezcan después de expirar.	En las ejecuciones de carga completadas: **0 duplicados**, revenue consistente, snapshots consistentes y **0 seats held** después de 3 TTL.
Adversarial	15%	Comportamiento bajo contención de sectores pequeños, últimos asientos, devolución de holds, expiraciones y sectores agotados.	Las pruebas incluyen estos escenarios y **no se observaron violaciones** de las propiedades de seguridad.

Liveness	10%	Que las operaciones terminen dentro del presupuesto y que los clientes no queden bloqueados o excluidos indefinidamente.	Las ejecuciones completadas **no presentaron clientes bloqueados**. Bajo 64 y 256 clientes se observó degradación importante y las ejecuciones experimentales superaron el límite de 180 s.
Scalability	10%	Capacidad de manejar mayor concurrencia y mantener independencia entre sectores.	La implementación mantiene independencia entre sectores, pero el throughput disminuye bajo alta contención: aproximadamente **527 req/s con 4 clientes**, **268 req/s con 16**, y ejecuciones parciales con 64 y 256.
Performance	10%	Throughput y latencia observados en las pruebas de carga realizadas.	Con 4 clientes: **527 req/s**, p50 **3.5 ms**, p99 **34 ms**. Con 16 clientes: **268 req/s**, p50 **3.9 ms**, p99 **648 ms**. La contención incrementa los conflictos y reintentos.

6. Conclusión

La implementación de `STM` reemplaza la exclusión mutua utilizada en la tarea anterior por un modelo de concurrencia optimista basado en versiones. Las operaciones pueden realizar su cálculo sobre un estado local y solo publican el resultado si el estado que utilizaron continúa siendo válido.

Las pruebas realizadas permiten verificar las principales propiedades de seguridad y consistencia, incluyendo reservas `all-or-nothing`, ausencia de ventas duplicadas o perdidas, manejo de holds, expiración, confirmación y cancelación, así como la consistencia de los snapshots.

La evaluación de rendimiento muestra que el costo del `STM` aumenta bajo alta contención, debido a los conflictos y reintentos necesarios para mantener la consistencia del estado. Por tanto, el modelo `STM` permite estudiar el intercambio entre concurrencia optimista, consistencia y costo de ejecución en un sistema de reservas concurrente.